QA Intern (RL Environments): Zero to Pro

Last-minute revision sheet for the technical round. Read Parts 1 to 4 for the concepts, Part 5 for this specific role, Part 6 for your assignment, and Part 7 for quick-fire answers. Every topic has a one-line answer you can say out loud.

How to use this in the last hours: (1) Read the one-line "Say it" boxes first. (2) Read Part 6 twice, because the assignment is the most personal thing they can ask about. (3) Rehearse Part 7 out loud. If you don't know something, say "I haven't worked with that yet, but here is how I'd approach it". That answer is fine for an intern role.

Part	What it covers	Priority
1	QA vs testing, SDLC vs STLC, levels of testing	High
2	Testing types, edge cases, test case vs scenario, manual vs automation	High
3	Bugs: life cycle, report fields, severity vs priority, category	Highest
4	Web app basics, REST API, status codes, Postman	High (Must Have in JD)
5	RL environments, graders, and what the company does	High (role-specific)
6	Your DocuSign-clone bug tracker: how to talk about it	Highest
7	Quick-fire Q&A, situational and HR questions, questions to ask	High

Part 1: The big picture

QA vs Software Testing

	Testing	QA (Quality Assurance)
Meaning	The activity of finding bugs	The whole process of building quality in
Focus	Finding defects	Preventing defects
When	After code is built	Throughout development
Nature	Reactive	Proactive

Analogy: testing is tasting the cake after baking. QA is checking the recipe, ingredients and oven before and during baking.

Say it: *Testing is a part of QA. Testing finds defects, while QA is the broader process that aims to prevent them.*

SDLC vs STLC

SDLC is the whole software creation process (plan, design, code, test, deploy, maintain). **STLC** is the testing part inside it. STLC has 6 phases:

#	Phase	What happens	Example
1	Requirement analysis	QA studies requirements and flags anything unclear	"Login should be fast": how fast?
2	Test planning	Scope, strategy, tools, roles, timeline, risks. Output: test plan	Decide what will and won't be tested
3	Test case development	Write test cases and prepare test data	Wrong password expects an error message
4	Environment setup	Prepare servers, browsers, devices, data. Often a smoke test here	Confirm the build opens
5	Test execution	Run tests, log bugs, retest fixes	Actual vs expected

#	Phase	What happens	Example
6	Test closure	Summary report, lessons learned, sign-off	Give sign-off on the environment

Memory trick: Read, Plan, Write, Setup, Run, Report. Each phase has **entry criteria** (what must be ready before it starts) and **exit criteria** (what must be done before it ends).

Say it: *STLC is the structured sequence of activities in testing: requirement analysis, planning, test case development, environment setup, execution and closure.*

Levels of testing (small to large)

Level	What it tests	Who usually does it
Unit	One small piece of code (a function) in isolation	Developers
Integration	Whether modules work together (cart + pricing)	Developers and QA
System	The whole application end to end, like a real user	QA
Acceptance (UAT)	Whether it meets business or client requirements	Client or QA for the client

For this role you are mostly doing system-level and acceptance-level testing on environments and graders.

Part 2: Types of testing

Functional vs non-functional

	Functional	Non-functional
Checks	Does it work correctly?	How well does it work?
Example	Does login accept valid credentials and reject invalid ones?	How fast is login? Can 1000 users log in at once?
Focus	Features and requirements	Speed, security, usability, load

Say it: *Functional testing checks what the system does against requirements. Non-functional testing checks how well it does it.*

Smoke vs sanity vs regression

Type	Question it answers	Scope	When
Smoke	Is this build stable enough to test at all?	Broad, shallow	After a new build
Sanity	Did this specific fix work?	Narrow, quick	After a small fix
Regression	Did this change break something old?	Deep, thorough	After any change

- **Smoke** example: app opens, login loads, homepage loads. If not, reject the build.
- **Sanity** example: forgot-password was fixed, so check only that flow.
- **Regression** example: a coupon feature was added, so re-check login, cart, checkout and totals. Usually automated because the same tests repeat every release.
- **Choosing regression tests:** impact analysis (what did the change touch), core features first, previously buggy areas, and a maintained regression suite.

Exploratory, usability and viewport testing

- **Exploratory:** no fixed script. You learn the app and test at the same time, using judgment. Best when documentation is thin.
- **Usability:** is it easy and clear to use? (labels, navigation, error messages). A feature can work and still be confusing.

- **Viewport / responsive:** does the layout still work on different screen sizes and orientations? Tools: browser DevTools device mode, BrowserStack.

Edge cases: 7 categories to think through

Category	Example
Boundary values	Age field accepts 18 to 60: test 17, 18, 60, 61
Empty / null	Submit a form with everything blank
Extreme values	500 characters in a 50-character field; a 10 GB upload
Invalid type or format	Letters in a phone field; Feb 30th
Special characters	Emojis, quotes, extra spaces
Timing and concurrency	Double-click Submit; two users book the last seat; internet lost mid-action
Sequence breaking	Press Back after payment; open a page URL without logging in

Say it: *Edge cases are unusual or extreme situations outside the normal flow. They are where bugs hide, so I test boundaries, blanks, extremes, invalid input and odd sequences.*

Test scenario vs test case

	Test scenario	Test case
Meaning	What to test (high level)	How to test it (detailed)
Example	Verify login works	Enter valid email and valid password, click Login, expect dashboard
Contains	One line	Steps, test data, expected result

Manual vs automation testing

	Manual	Automation
Speed	Slow	Fast
Best for	Exploring, usability, new features	Regression, smoke, repeated and data-heavy checks
Cost	Cheap to start, costly over time	Costly to set up, cheap to repeat
Finds	Unexpected problems	Only what the script checks

- **Automate:** regression, smoke, data-heavy, stable features.
- **Don't automate:** exploratory, usability, one-time tests, features that change constantly.
- **Tools (know the names):** Selenium, Playwright, Cypress (web UI); Postman, RestAssured (API); TestNG, JUnit, PyTest (frameworks).

Say it: *Automation testing runs tests with scripts. It suits repetitive tests like regression and smoke testing, while manual testing suits exploratory and usability work. Both are needed.*

Part 3: Bugs (the heart of this role)

Error vs defect (bug) vs failure

In daily work, **bug** and **defect** are used interchangeably (Jira's issue type is "Bug"; some enterprise tools say "Defect"). The textbook chain is:

Term	Meaning	Example
Error	A human mistake	Developer misreads the requirement

Term	Meaning	Example
Defect / bug	The flaw in the code caused by that error	Wrong logic in the price calculation
Failure	What the user sees when the flaw is triggered	The cart shows the wrong total

Bug life cycle

New -> Assigned -> In Progress -> Fixed -> Retest -> Verified -> Closed. Side branches: **Reopened** (still broken after retest), **Rejected** (not a bug or expected behavior), **Deferred** (valid but fixed later), **Duplicate** (already logged).

What a good bug report contains

Field	What to write
Title	One clear line: what is wrong and where
Steps to reproduce	Numbered steps anyone can follow exactly
Expected vs actual	What should happen vs what happened
Evidence	Screenshot or video (filename in the tracker)
Severity and priority	Impact and urgency (see below)
Environment	Browser, OS, viewport, build version
Reproducibility	Always, sometimes or once

Good title: "[Inbox] Table overflows viewport, Status column cut off". Bad title: "Inbox broken".

Category: how it works or how it looks?

- **Functional:** something doesn't work or gives wrong results (dead button, wrong total, grader marks a wrong answer as Pass).
- **Visual / UI:** it works but looks wrong (overlap, misalignment, wrong color, cut-off text).
- Others you may see: usability, performance, data, content/text (unclear task prompt), compatibility.

Severity: your tracker's own scale (P0 to P4)

Tag	Name	What it means (from the tracker's legend)
P0	Blocker	Stops the user cold. App crash, data loss, login broken, major flow unusable
P1	Critical	Major feature broken for many users but a workaround exists, or severe UI breakage on a core screen
P2	Major	Functional bug on a common flow that's ugly or confusing but doesn't block. Or a pixel issue on a primary screen
P3	Minor	Edge-case bug, small visual issue, uncommon path. Low user impact
P4	Trivial	Cosmetic, typo, 1px spacing issue. Worth noting but low priority

Quick decision: crash or blocked with no way around it = P0. Main feature wrong = P1. Works but confusing, or a layout problem on a main screen = P2. Small or rare = P3. Cosmetic = P4.

Do not mix these up. In your tracker, P0 to P4 is the **Severity** scale. The **Priority** column uses High, Medium and Low. Some companies use P0 to P4 for priority instead, so in an interview say which scale you mean.

Priority: how urgently to fix (High / Medium / Low)

The company's legend defines severity only, so priority below is a common convention, not their rule:

Severity	Usual priority	Why
P0, P1	High	Blocks work or a major feature is broken
P2	Medium	Visible and annoying, but work continues
P3, P4	Low	Small, rare or cosmetic

Severity vs priority

	Severity	Priority
Question	How bad is the damage?	How soon must it be fixed?
Decided by	QA (you)	Lead or product owner (QA suggests)
Based on	Technical impact	Business need, visibility, deadlines

They can differ. High severity, low priority: a crash in a feature almost nobody uses. Low severity, high priority: a typo in the company name on a homepage the day before a client demo.

> ***Say it:*** *Severity is the technical impact of a bug. Priority is how urgently the business needs it fixed. QA usually sets severity and the lead decides priority.*

Part 4: Web apps and REST APIs

How a web app works

- **Frontend:** what you see and click (buttons, pages), running in the browser.
- **Backend:** the server logic that processes requests and applies the rules.
- **API:** the messenger between them. The frontend sends a request, the backend sends a response.
- **Database:** where the data is stored. "Data seeding" means pre-loading sample data so testers have something to work with.

Flow: you click a button, the frontend sends an API request, the backend reads or writes the database, the backend returns a response, and the frontend shows the result. A bug can live in any of these layers.

REST API basics

Method	Purpose	Example
GET	Fetch data	Get one post
POST	Create data	Create a new post
PUT / PATCH	Update data (PUT replaces, PATCH changes part)	Edit a post
DELETE	Remove data	Delete a post

Code	Meaning
200	Success
201	Created
400	Bad request (client sent something invalid)
401	Unauthorized (missing or invalid login)
404	Not found
500	Internal server error

- **An API test checks:** the right status code, the right response body, and behavior on bad input (missing fields, wrong types, no authorization).

- **Postman** is used to send requests manually and inspect responses. **RestAssured** is a Java library for automated API tests.

10-minute Postman practice (do it before the interview)

- GET https://jsonplaceholder.typicode.com/posts/1 and note the 200 and the JSON body.
- GET .../posts/9999 and note the 404.
- POST https://jsonplaceholder.typicode.com/posts with a small JSON body and note the 201.

Say it: *I've practiced sending GET and POST requests in Postman and checking status codes and response bodies.*

Part 5: RL environments, graders and the company

What is an RL environment?

Reinforcement learning (RL) trains an AI by letting it attempt tasks and giving feedback on how well it did. An **RL environment** is a simulated workspace (often a copy of a real tool like a CRM or a document-signing app) with **tasks** the AI must complete. A **grader** (also called a verifier) is automated code that checks whether the AI's attempt was correct and marks it pass or fail.

Why it needs QA: if a task is unclear, or the grader is wrong, the AI learns from bad feedback. A grader that passes wrong answers (false pass) or fails right ones (false fail) makes the whole environment untrustworthy.

How to test a grader

- Give it a clearly correct answer: it should pass.
- Give it a clearly wrong answer: it should fail.
- Give it edge cases: blank answer, very long answer, wrong format, a valid answer done in an unusual way.
- Run the same input more than once: the result should be identical.
- Look for ways to cheat it (hard-coding the expected output) and report them.

A good task prompt is

- **Clear:** one meaning only.
- **Complete:** has all information needed.
- **Completable:** it can actually be done inside the environment.

About the company (from their website, Sep 2026)

"Scaler AI Labs" is now **Evaratus AI**. Per evaratus.com, they build RL environments ("RL gyms"), evaluations, enterprise data and applied research to help frontier AI models get better at real-world work. Their site describes a loop of failure analysis, task design, verification and reward modeling, and says changes ship only after measured improvement on held-out tasks. Co-founders named on the site: Anshuman Singh and Naman Bhalla.

Say it: *Since Evaratus only ships changes after proving measured improvement through verification, QA on the environments and graders protects whether those results can be trusted.*

How the JD maps to what you learned

JD line	Where it fits
Verify prompts are clear and completable	Requirement analysis, usability of prompts
Test UI components, workflows, edge cases against scope docs	Functional, exploratory and edge-case testing
Log bugs with steps, screenshots, severity	Bug report fields, severity vs priority
Validate fixes across multiple rounds	Retest, regression, bug life cycle
Test grader logic for pass/fail	Part 5 grader tests

JD line	Where it fits
Give sign-off	Test closure, exit criteria

Part 6: Your assignment (the DocuSign-clone bug tracker)

Your tracker has 31 bugs found by comparing a clone against the original. Expect to be asked about it. Prepare these four answers.

1. How did you find the bugs?

Say it: *I compared the clone against the original screen by screen, checked buttons, menus and navigation, looked at layouts on different views, and then tried edge cases and unusual paths.*

2. Walk me through one bug

Pick one you know well. Say it in this order: **where** (screen), **steps**, **expected**, **actual**, **category**, **severity and why**, **priority and why**, **evidence**. Example bugs from your sheet:

Bug	Category	Severity	Priority	Why (say this)
BUG-017 Inbox table overflows viewport	Visual / UI	P2	Medium	Matches the legend: pixel issue on a primary screen. Data is still reachable by scrolling, so not urgent.
BUG-026 "Send with AI" button does nothing	Functional	P2	High	Other send routes work, so P2. Every user sees the button on Home, so priority is High.
BUG-011 Reports and Admin tabs disabled	Functional	P1	High	Two whole sections are unusable, but the main flows work, so P1 not P0.
BUG-007 Footer text alignment	Visual / UI	P4	Low	Pure cosmetic spacing.

3. Why that severity and not another?

Use the three questions: does it block the user (P0)? Is a main feature wrong (P1)? Is it confusing or a layout issue on a main screen (P2)? Otherwise it is minor (P3) or cosmetic (P4). Always say you follow the team's severity guide.

4. What would you fix in your own tracker?

- **Clean trailing spaces** in some Severity cells (for example "P3 ") and one Bug ID. They can break dropdowns and filters.
- **Check consistency between similar bugs.** Controls that do nothing should get the same severity (BUG-020 is P4 while the similar BUG-022, 023 and 027 are P3).
- **Reconsider a few ratings:** BUG-013 (Tasks and Help icons) as P1 looks high for secondary features, and BUG-031 (most field types and actions disabled on the tagging screen) may deserve P1.
- **One row = one bug.** A few rows bundle several issues (for example, three disabled menu items in one row). Splitting them makes tracking cleaner.
- **Verify BUG-005** in a fresh (incognito) session, because your notes say it may be an account setting.

Say it: *On review I found a few inconsistencies in my own tracker and here is how I'd fix them. I'd rather catch that than leave it.*

Part 7: Quick-fire answers

Technical

Q: What is the difference between QA and testing?

Testing finds defects. QA is the wider process that prevents them. Testing is part of QA.

Q: Smoke vs sanity?
Smoke checks whether a new build is stable enough to test (broad, shallow). Sanity checks whether one specific fix works (narrow, quick).

Q: What is regression testing?
Re-testing existing features after a change to make sure nothing that worked before broke. Usually automated.

Q: What is exploratory testing?
Testing without a fixed script, learning the app and designing tests on the go using judgment.

Q: What is an edge case?
An unusual or extreme scenario outside normal usage, like blank input, boundary values or a double click, where bugs often hide.

Q: Severity vs priority?
Severity is technical impact. Priority is urgency. QA sets severity, the lead sets priority. They can differ.

Q: Bug vs defect?
Used interchangeably in practice. Textbooks say a bug is found in testing and a defect is a deviation from requirements. An error is the human mistake that causes them.

Q: Explain the bug life cycle.
New, Assigned, In Progress, Fixed, Retest, Verified, Closed, with Reopened, Rejected, Deferred and Duplicate as side branches.

Q: What is STLC?
The six testing phases: requirement analysis, test planning, test case development, environment setup, test execution and test closure.

Q: What are the HTTP methods and common status codes?
GET fetch, POST create, PUT or PATCH update, DELETE remove. 200 success, 201 created, 400 bad request, 401 unauthorized, 404 not found, 500 server error.

Q: What would you automate?
Repetitive, stable tests such as regression, smoke and data-heavy checks. Not exploratory or usability testing.

Q: What is a grader and how would you test it?
An automated check that marks an AI's attempt pass or fail. I'd test a correct answer, a wrong answer, blank and unusual answers, and repeat runs, and report any false pass or false fail.

Situational (approach: understand, act step by step, communicate)

Q: A developer says your bug is not a bug.
I'd re-verify it, share exact steps, expected vs actual and evidence, check the spec, and involve the lead if we still disagree.

Q: Deadline is close and there are many tests left.
I'd test high-risk, core flows first, and clearly tell my lead what was covered and what wasn't.

Q: The grader passes a wrong answer.
I'd record the exact input, check whether it is a pattern, and report it to the developers as high priority because grader accuracy is the core of the environment.

Q: You test something with no documentation.
I'd explore it like a user first, note anything inconsistent, and ask clarifying questions instead of guessing what correct means.

Q: You find a bug outside your assigned area.
I'd still log it with steps and evidence and let the lead decide its priority.

HR and closing

Q: Tell me about yourself.

About 60 seconds: your degree, what you learned in QA and API testing, one thing you did for the assignment, and why this role interests you (testing AI systems is new and exciting).

Q: Why QA?

I enjoy noticing what others miss and thinking about what could go wrong, and QA combines technical thinking with clear communication.

Q: Why should we hire you?

Attention to detail, a structured but independent approach to edge cases, clear bug reports, and fast learning. Mention the assignment as proof.

Q: Weakness?

I'm still building depth in automation frameworks, so I'm practicing with Postman and basic Selenium.

Q: How do you handle pressure and shifting priorities?

I list tasks by impact, communicate early if something will slip, and stay accurate under pressure.

Questions you can ask them

- What does a typical day look like in this QA role?
- How is feedback given across the QA rounds?
- What does success look like in the first three months?

Last-hour checklist

- Say the one-line answers for: QA vs testing, smoke vs sanity vs regression, severity vs priority, bug life cycle, STLC, grader testing.
- Have 2 bugs from your tracker ready to walk through (suggested: BUG-017 and BUG-026).
- Know the P0 to P4 severity legend and that priority is High, Medium, Low.
- Know the HTTP methods and the six status codes.
- Know what Evaratus does in one sentence.
- Do the 10-minute Postman practice if you haven't.
- Sleep, water, and log in to the interview link a few minutes early.

Remember: for an intern role they are testing how you think, how clearly you communicate, and whether you stay calm. A structured honest answer beats a memorized perfect one.